Package 'mdi'

March 26, 2026

Type Package

Title Microdata Infrastructure: tools for statistical agencies and users at their research data centers

Version 0.2.1

License GPL-3

Author Eric Bartelsman, with contributions by Jing Chen, Mirja Halbig, Alessandro Zona Mattioli, Daniele Aglio, Marco Miorandi, Johanna Weiss, Reetuparna Vishwanath

Maintainer Comp-net Org <info@comp-net.org>

Imports data.table

Description The mdi package includes tools for statistical agencies (NSIs) and users at their research data centers. The tools are divided in four main categories:
- Infra: tools used to set up the NSI environment.
- MDI: general tools for NSIs and MDI users for purposes such as data aggregation, maniupulation etc.
- Analysis: research-related tools, used or statistical analysis.
- Programmer: additional tools including metadata handling, general hierarchical mappings etc.

Encoding UTF-8

LazyData true

RoxygenNote 7.3.3

Contents

build_firm_dimensions . . . 2
compute_all_weights . . . 3
concord_nace . . . 4
conc_type . . . 5
consolidate_output . . . 6
cs_prodest . . . 7
disclosCrit . . . 8
disclose . . . 9
disclose_reg_tab . . . 10
estimate_markup . . . 11
estimate_outputelast . . . 12
estimate_prod . . . 13
gg_hist_density . . . 14
gg_scatter_reg . . . 15
harmonize_classification . . . 16

hier_apply 17
import_data 18
log_warnings 19
make_conc 20
make_filelist 20
mdi_aggregate 21
mdi_disagg 22
mdi_estimate_prodfun 23
mdi_export 27
mdi_harmonize 29
mdi_import_mpnames 29
mdi_jointdist 30
mdi_key_id_switch 31
mdi_mergedatatables 32
mdi_outlier 33
mdi_regress 34
mdi_relpath 36
mdi_sampleweights 37
md_cp2hier 38
pim_capital 38

`build_firm_dimensions` *Build Firm-Level Dimension Variables (Size, Age, Industry)*

Description

Adds three categorical dimensions to a BR-style dataset (real or mock): firm size (‘dim_1_sc‘), firm age (‘dim_2_age‘), and industry (‘dim_3_ind‘, based on 2-digit NACE). The function expects person counts, birth dates, and NACE codes in the dataset and merges in a NACE hierarchy table.

Usage

```
build_firm_dimensions(DT, MD_nace_hier, id_col = "entid")
```

Arguments

`DT`	A data.frame or data.table containing firm-level observations. Must include: • ‘persons_br‘ — number of persons employed. • ‘birthyr‘ — firm birth date as string in ‘ • ‘year‘ — reference year of the observation. • ‘MDnace‘ — NACE code matching ‘MD_nace_hier$h_0‘.
`MD_nace_hier`	A data.table containing a NACE hierarchy with at least columns: • ‘h_0‘ — original NACE code. • ‘h_2‘ — corresponding 2-digit NACE aggregation.
`id_col`	String. Column name identifying firms (default ‘"entid"‘).

Details

* Size classes follow standard employment thresholds. * Age is computed as ‘year - founding_year‘, where the founding year is the earliest observed birth year across observations of each firm. * Industry codes are merged from ‘MD_nace_hier‘.

Value

A data.table equal to ‘DT‘ with three added variables:

- ‘dim_1_sc‘ — firm size class: ‘"micro"‘, ‘"small"‘, ‘"medium"‘, ‘"large"‘.
- ‘dim_2_age‘ — age class: ‘"1_4"‘, ‘"5_9"‘, ‘"10_15"‘, ‘"20_"‘.
- ‘dim_3_ind‘ — 2-digit NACE industry.

The function also removes internal temporary variables.

Examples

```
## Not run:
enriched <- build_firm_dimensions(DT, MD_nace_hier)

## End(Not run)
```

---

compute_all_weights *Compute Disaggregation Weights Automatically*

Description

Computes disaggregation weights between hierarchical IDs (e.g., plant → firm) using available lower-level datasets automatically detected from ‘MD_idInfo‘. For each relevant numeric variable (e.g., ‘rev‘, ‘nv‘, ‘emp‘, ‘pay‘, ‘ntrade‘, ‘total_assets‘), the function aggregates data at the lower and upper ID levels using the internal ‘mdi_aggregate()‘ function, computes proportional weights, and returns a combined weight table.

Usage

```
compute_all_weights(conctab)
```

Arguments

conctab	‘data.table‘ Concordance table mapping lower- and upper-level IDs (e.g., ‘plantid‘ -> ‘firmid‘).

Details

The function reads all datasets corresponding to the lower-level ID as specified in ‘MD_idInfo‘ for the given country. It identifies suitable numeric variables available in those datasets and computes weights by proportionally distributing each variable’s total value across the upper-level ID (e.g., distributing firm-level SBS totals using plant-level PRODCOM data).

If no valid numeric variables are found, the function defaults to equal weights within each group.

Value

A ‘data.table‘ containing:

- The lower-level ID
- The upper-level ID
- The ‘year‘ variable
- Weight columns of the form ‘varname_w‘, one for each computed variable

Author(s)

Marco Miorandi, 2025

Examples

```
## Not run:
conctab <- fread(file.path(dirNSIMETA, "SI_plantid_firmid_conc.csv"))
weights <- compute_all_weights(conctab)
head(weights)

## End(Not run)
```

`concord_nace` *NACE classification harmonization*

Description

This tool harmonizes NSI NACE classification over time. It simplify the harmonization process using the mode NACE code in the post break year as MD NACE code, and harmonizing accordingly previous year codes. It defines the break year as the year with the most changes in NACE codes.

Usage

```
concord_nace(dataset_name, weight_emp, cumsh_del, sh_del)
```

Arguments

`dataset_name:`	character to indicate the dataset name;
`weight_emp:`	logical to decide whether or not to weight code matches by employment (instead of number of firms).
`cumsh_del:`	number indicating the cumulative residual share of firms deleted for the construction of year by year concordances.
`sh_del:`	number indicating the share of firms deleted for the construction of year by year concordances.

Value

the original MD dataset with the old NSI NACE code and the standard MD NACE.

Author(s)

Daniele Aglio and Eric Bartelsman, 22/10/2024.

Examples

```
br_harmonized <- concord_nace_simple("br", FALSE, 0.95, 0.02)
```

conc_type *Classify the concordance rows (edges)*

Description

This tool classifies concordance edges into type 1:1, 1:m, m:1, or m:m. Optionally, the tool also creates numbered 'components', or block-diagonals of the m:m edges. The input is a concordance data.table, with left and right columns and the function returns a data.table, possibly merged with the initial table, with conctype (and optional compnum).

Usage

```
conc_type(
  DTconc,
  left = "left",
  right = "right",
  mm_groups = TRUE,
  mergeconc = FALSE
)
```

Arguments

DTconc:	a concordance table (data.table) with two columns: given by parameters left and right.
left:	character ('left') is the name of the left side of the concordance
right:	character ('right') is the name of the right side of the concordance
mm_groups:	logical (TRUE) if m:m grouping (block diagonalization) is required.
mergeconc:	logical (FALSE) returns the conc merged with the original input

Value

Returns a data.table[‘left‘, ‘right‘,conctype, outdegree, indegree, compnum] where conctype gives type of match ("1:1","m:1","1:m","m:m"); outdegree gives number of right nodes to which a left node goes; indegree gives number of left nodes that go to a right node; if mm_groups=TRUE, then ‘compnum‘ gives component (block-diagonal) number. If mergeconc=TRUE then conc_type() returns the result merged with the original DTconc input.

Author(s)

Daniele Aglio and Eric Bartelsman, version Sep 22, 2024.

Examples

```
concx <- data.table(
  leftx = as.character(c(1, 2, 2, 3, 4, 5, 6, 6, 7, 8, 8)),
  rightx = c("A", "B", "C", "D", "D", "F", "E", "F", "H", "G", "H")
)
conc_type <- conc_type(concx, left = "leftx", right = "rightx", mm_groups = TRUE, mergeconc = TRUE)
```

`consolidate_output` *Stack Output Function*

Description

This function stacks output datasets based on the specified module name. It performs the following operations: - Reads a CSV file listing the outputs for each module, - Filters the list for the selected module, - Reads in the relevant files for each country where the file is available and appends data for different filenames, - Adds a country identifier to each dataset, - Stacks these datasets together.

This function assumes the existence of a CSV file that lists the outputs for each module. It requires the setup of the following directories: 'input_directory', 'output_directory', and 'metadata_directory'.

tooltype: Infra

Usage

```
consolidate_output(module_name, metadata_file, base_path, output_directory)
```

Arguments

`module_name`	The abbreviation of the module name (e.g., EN, FD) as specified in the metadata file.
`metadata_file`	full path of metadata file which contains the full list of outputs for all countries. Necessary columns: module, moduleabbrev, generic_filename, and country dummies indicating whether the file is present in that country
`base_path`	base directory where all country outputs are saved on Teams, from which the country-specific dynamic paths will be constructed. Should be of the form: /MDI Data Providers Forum - COUNTRY - COUNTRY/Upload/Launchv2.1
`output_directory`	main directory where the stacked outputs should be saved.

Value

An invisible data frame that combines all the stacked datasets. The function also saves this data frame as a CSV file in the 'output_directory' directory.

Author(s)

Reetuparna Vishwanath

Examples

```
Define file paths
metadata_file <- "/Users/reetuparnavishwanath/Documents/GitHub/MDI/payload/Launch_v2.1/Results/launch_2.1_f
base_path <- "/Users/reetuparnavishwanath/Library/CloudStorage/OneDrive-SharedLibraries-IWHEconomicStudiesL
output_directory <- "/Users/reetuparnavishwanath/Documents/GitHub/MDI/payload/Launch_v2.1/Results"
Run the function for all modules
modules <- c("EN", "FD", "TC", "MP", "CN", "M0")
for (module in modules) {
  consolidate_output(module, metadata_file, base_path, output_directory)
}
```

`cs_prodest` *Cost-shares production-function estimator (Cobb–Douglas)*

Description

Implements cost-shares approach as in the deprecated routine: - Build firm-time shares from input expenditures in LEVELS - Average shares by (bygroup, time) unless time is already in the grouping set - Compute TFP as log-index residual using averaged shares - Demean TFP by the same (bygroup, time) group unless time is already included

Implements cost-shares approach as in the deprecated routine: - Build firm-time shares from input expenditures in LEVELS - Average shares by (bygroup, time) unless time is already in the grouping set - Compute TFP as log-index residual using averaged shares - Demean TFP by the same (bygroup, time) group unless time is already included

Usage

```
cs_prodest(DT, y, endog, exog, id, time, bygroup, log_values = TRUE)
```

```
cs_prodest(DT, y, endog, exog, id, time, bygroup, log_values = TRUE)
```

Arguments

`DT`	data.table/data.frame with panel data
`y`	character scalar: output variable name
`endog`	character scalar: free input (expenditure) variable name
`exog`	character scalar: state input (expenditure) variable name
`id`	character scalar: entity id
`time`	character scalar: time id
`bygroup`	character scalar: grouping variable (e.g., industry)
`log_values`	logical; default TRUE. If TRUE, y/endog/exog are logs and are exponentiated before share construction.

Value

data.table with columns: - id, time, bygroup - el_endog, el_exog - tfp - NumObs (within share-averaging group after CS filtering)

data.table with columns: - id, time, bygroup - el_endog, el_exog - tfp - NumObs (within share-averaging group after CS filtering)

`disclosCrit` *Add Disclosure Criteria to Aggregated Data*

Description

Computes and attaches disclosure-control variables to an aggregated dataset. The function calculates a dominance measure (share of the top ‘domNr‘ firms in terms of employment, sales, or all variables in ‘var_list‘) and the number of non-missing observations per group.

This function is normally called internally by [‘mdi_aggregate()‘] when ‘disclosure = TRUE‘, but can also be used standalone.

tooltype: MDI

Usage

```
disclosCrit(
  DT,
  domVar = MDIdomVar,
  domNr = MDIdomNr,
  bygroups,
  var_list = var_list
)
```

Arguments

`DT`	A ‘data.table‘ containing the aggregated dataset.
`domVar`	Character. Variable used for the dominance criterion: - ‘"emp"‘: employment, - ‘"nq"‘: sales, - ‘"var"‘: all variables specified in ‘var_list‘. Default is taken from ‘MDIdomVar‘.
`domNr`	Numeric. Number of top firms to consider in the dominance criterion (e.g. top 1, 2, or 3). Default is taken from ‘MDIdomNr‘.
`bygroups`	Character vector of grouping variables, as in [‘mdi_aggregate()‘].
`var_list`	Character vector of variables to include when ‘domVar = "var"‘. Usually the same as in [‘mdi_aggregate()‘].

Details

- For ‘domVar = "emp"‘ or ‘domVar = "nq"‘, only one dominance column (‘domPerc‘) is returned. - For ‘domVar = "var"‘, separate dominance columns are created for each variable in ‘var_list‘ (‘domPerc_<var>‘). - For Germany (‘CountryCode == "DE"‘), a specific dominance formula is applied:

$$(Total - x_1 - x_2)/x_1$$

where x_1 and x_2 are the largest two values.

Value

A ‘data.table‘ with the same grouping structure as the input, plus: - One or more ‘domPerc_*‘ columns: share of the top ‘domNr‘ firms per group. - A column ‘NumObs‘: the number of non-missing observations for the corresponding dominance variable(s).

Author(s)

Marco Miorandi, 22/09/2025 (last changes)

Examples

```
# Example: dominance share of top 2 firms in employment
DTdisc <- disclosCrit(DT, domVar = "emp", domNr = 2, bygroups = c("nace","year"))

# Example: dominance shares for all variables in var_list
DTdisc <- disclosCrit(DT, domVar = "var", domNr = 3,
                      bygroups = c("nace","year"),
                      var_list = c("emp","rev"))
```

---

disclose *Disclosure Check for Aggregated Data*

Description

Applies disclosure control rules to aggregated data based on dominance and minimum-observation criteria. This function is typically called in [‘mdi_export()‘] for type ‘"sum_stats"‘.

Two steps are performed: - **Primary disclosure:** Suppresses (‘-999‘) all aggregate cells that fail disclosure rules: (i) do not satisfy the dominance criterion (‘domPerc‘), or (ii) have fewer than ‘minNumObs‘ observations. Flags are recorded as ‘discflag1_*‘ and ‘discflag1‘.

- **Secondary disclosure:** If hierarchical aggregation (‘hier = TRUE‘) is used and exactly one child cell is suppressed within a parent group, the smallest remaining cell in that group is also suppressed (‘-999‘). Flags are recorded as ‘discflag2‘.

Usage

```
disclose(
  DT,
  hhfile = hhfile,
  hier = hier,
  minNumObs = MDIminNumObs,
  domSh = MDIdomSh
)
```

Arguments

DT	A ‘data.table‘ containing aggregated statistics and disclosure variables (e.g. from [‘mdi_aggregate()‘]).
hhfile	Character. Name of the hierarchy file used for aggregation (only relevant if ‘hier = TRUE‘ and multiple levels of aggregation exist).
hier	Logical. Set to ‘TRUE‘ if multiple aggregation levels (child/parent nodes) are present in the dataset. Default: same as input.
minNumObs	Numeric. Minimum number of observations required per cell, country-specific. Default: ‘MDIminNumObs‘.
domSh	Numeric. Threshold for maximum allowed dominance share, country-specific. Default: ‘MDIdomSh‘.

Details

- For Germany (‘CountryCode == "DE"‘), the dominance rule is inverted compared to other countries, as the dominance percentile formula is different. - Suppression affects both the target aggregate columns and, when triggered, the ‘NumObs‘ column. - Secondary disclosure ensures that no single cell can be back-calculated from its parent total.

Value

A ‘data.table‘ identical in structure to the input, but with: - Suppressed cells (‘-999‘) where disclosure criteria are not met. - Flags ‘discflag1_*‘ and ‘discflag1‘ for primary disclosure. - Flag ‘discflag2‘ for secondary disclosure (if applicable).

Examples

```
# Apply disclosure to aggregated data (no hierarchy)
DTdisc <- disclose(DTagg, hier = FALSE, minNumObs = 3, domSh = 0.85)

# Apply disclosure with hierarchical aggregation
DTdisc <- disclose(DTagg, hhfile = "nace_hierarchy.csv", hier = TRUE)
```

---

disclose_reg_tab *Disclosure Check and Masking for Regression Tables*

Description

Applies disclosure control rules to regression output tables. Rows that do not meet minimum thresholds for degrees of freedom, number of observations, or (for Germany) number of firms are flagged and sensitive regression statistics are masked.

Usage

```
disclose_reg_tab(DT, min_obs = MDIminNumObs, show_disclosed = FALSE)
```

Arguments

DT	A ‘data.table‘ containing regression output. Must include the columns ‘"df"‘, ‘"NumObs"‘, and ‘"coef"‘. For Germany (‘CountryCode == "DE"‘), it must also include ‘"NumFirms"‘.
min_obs	Numeric. Minimum threshold used for disclosure checks. Typically ‘MDIminNumObs‘.
show_disclosed	Logical. If ‘TRUE‘, disclosed values are shown even when flagged. If ‘FALSE‘ (default), disclosed values are masked with ‘NA‘.

Details

- **Germany (‘CountryCode == "DE"‘):** disclosure is based solely on ‘NumFirms < min_obs‘. - **Other countries:** disclosure is triggered when either ‘df < min_obs‘ or ‘NumObs < min_obs‘. - The following regression statistics may be masked if disclosure applies: ‘"Estimate"‘, ‘"Std. Error"‘, ‘"z value"‘, ‘"Pr(>|z|)"‘, ‘"ci.lower"‘, ‘"ci.upper"‘, ‘"R2"‘, ‘"AdjR2"‘, ‘"AIC"‘, ‘"BIC"‘, ‘"LogLik"‘.

Value

A list with three elements:

DT	A ‘data.table‘ of the regression output with disclosure rules applied. Masked cells are set to ‘NA‘ (unless ‘show_disclosed = TRUE‘).
vars	A character string listing the disclosed coefficient names (for use in output description). If all rows are masked, returns ‘"No coefficients disclosed (all masked)"‘.
redacted_n	An integer count of the number of rows flagged and masked.

Examples

```
# Apply disclosure rules to a regression table
result <- disclose_reg_tab(reg_table, min_obs = 3)
masked_DT <- result$DT
cat("Redacted rows:", result$redacted_n, "\n")
cat("Disclosed coefficients:", result$vars, "\n")

# Keep disclosed values visible (for debugging)
result <- disclose_reg_tab(reg_table, min_obs = 3, show_disclosed = TRUE)
```

---

estimate_markup *Markup estimation*

Description

The function ‘estimate_markup‘ estimates firm level markup based on De Loecker (2012) methodology.

tooltype: Analysis

Usage

```
estimate_markup(DT, oe = "oe_l", rev_col = "nq", input_cost = "nm")
```

Arguments

DT	input dataset
oe	output elasticity of the production function for a given input
input_cost	total cost of the given input. For example, for labour use ’Personnel costs’ (pay), for intermediate input ’Total purchases of goods and services’ (nm) and for energy ’Purchases of energy products (in value)’ (nnrg) Edit: Marcelo Ribeiro 26/06/25
rev	total revenues (turnover)

Value

The function returns a new variable with markup.

`estimate_outputelast` *Elasticity computation tool*

Description

This function 'estimate_outputelast' takes as input the regression output from a production function estimation and computes firm level elasticities. This computation changes according to the estimation method, where the possible methods are the same as in PilotProd: - OLS: implements a OLS residual with year FE and interactions with each input; - ACF: Ackerberg-Caves-Frazer (2015) methodology; - OP: Olley-Pakes (1996) methodology; - LP: Levinshon-Petrin (2003) methodology.

tooltype: Analysis

Usage

```
estimate_outputelast(
  DT,
  reg_output,
  l = "lnemp",
  k = "lnk_tan",
  m = "lnrm",
  years,
  method = "ols"
)
```

Arguments

`DT`	database where to add the new columns containing the elasticities
`reg_output`	regression output
`l`	input variable
`k`	input variable
`m`	input variable
`years`	list of years. This should be inputted in the same format as it would come out of unique(br_sbs$year) for instance
`method`	regression method

Value

The function returns one new variable per input, indicating elasticities: oe_k, oe_l, oe_m.

`estimate_prod` *Productivity Estimation Tool*

Description

This tool estimates productivity, allowing for multiple specifications of the production function and estimation methodologies. It can return input elasticities, TFP, or both, depending on the selected output_type. The function supports OLS, ACF, and OP methods, with translog (tl) or Cobb-Douglas (cd) specifications for the production function.

tooltype: Analysis

Usage

```
estimate_prod(
  DT,
  y,
  exog,
  endog = NULL,
  instr = NULL,
  fe = NULL,
  bygroups,
  id = NULL,
  time = NULL,
  method = "ols",
  spec = "cd",
  output_type = "ALL"
)
```

Arguments

`DT`	The input data.table containing all necessary variables for the estimation.
`y`	The output variable name. For ACF and OP methods, use value added.
`exog`	A vector of exogenous/state input variable names.
`endog`	Optional vector of endogenous/free input variable names.
`instr`	Optional vector of instrumental/proxy variable names.
`fe`	Optional vector of control/factor variable names for fixed effects.
`bygroups`	A vector of variables by which production functions are estimated separately.
`id`	Required for ACF and OP methods. Identifies the firm or entity.
`time`	Required for ACF and OP methods. Identifies the time period.
`method`	The estimation method: 'ols', 'acf', 'op'.
`spec`	The specification of the production function: 'tl' for translog or 'cd' for Cobb-Douglas. For OP, only 'cd' is allowed.
`output_type`	The desired output_type: 'el' for only elasticities, 'tfp' for TFP estimation only, or 'ALL' for both.

Value

A data.table column with the requested output: input elasticities and/or TFP, based on the output parameter selection.

Author(s)

Cindy Jing Chen, revised by Mirja Haelbig and further revised by Alessandro & Mirja

Examples

```
# OLS estimation for translog specification
cols <- c('el_k', 'el_l', 'el_m', 'tfp')
result <- estimate_prod(DT = br_sbs, y = 'lnrv', exog = c('lnk_tan', 'lnemp', 'lnrm'),
                  fe = 'year', bygroups = 'h_2', method = 'ols', spec = 'tl')

# ACF estimation for Cobb-Douglas specification
cols <- c('el_k', 'el_l', 'tfp')
result <- estimate_prod(DT = copy(db), y = 'lnrv', exog = 'lnk_tan', endog = 'lnemp', instr = 'lnrm',
                  fe = 'year', bygroups = 'h_2',
             id = 'firmid', time = 'year', method = 'acf', spec = 'cd', output_type = 'ALL')
```

---

gg_hist_density *Histograms*

Description

General tool for producting nice histograms.

tooltype: Analysis

Usage

```
gg_hist_density(
  DT,
  var_name = "nq",
  trim = NULL,
  both_tails = FALSE,
  density = TRUE,
  mean = TRUE,
  x_name = var_name,
  title = NULL
)
```

Arguments

DT	a ‘data.table‘ that contains the variable
var_name	variable name
trim	if NULL, no trimming, otherwise specify a pctile and the chart will plot only the observations below that pctile
both_tails	if TRUE, trimming is done also on the lower tail. Default is FALSE
density	fit a density over the histogram
mean	if TRUE, add a dashed line with mean to histogram
x_name	set name for x_axis. Default is var_name
title	set title of histogram. Default is ’Histogram of ’ + x_name

Value

It displays the resulting graph. It must be saved manually as saving options are not included.

`gg_scatter_reg` *(Analysis tool) Make x-y scatter plot and draw regression line.*

Description

'gg_scatter_reg()' displays the graph. Future updates with allow writing in in various formats.

Usage

```
gg_scatter_reg(
  DT,
  var_names,
  trim = NULL,
  both_tails = FALSE,
  reg_line = "lin",
  reg_weight = NULL,
  x_name = NULL,
  y_name = NULL,
  title = NULL
)
```

Arguments

`DT`	A data table containing the data.
`var_names`	character list (EB: is this right?) with the two (X,Y) variable names to compute the correlation/regression.
`trim`	integer if NULL, no trimming, otherwise specify a pctile and the chart will plot only the observations below that pctile. Default is NULL
`both_tails`	Boolean if TRUE, trimming is done also on the lower tail. Default is FALSE.
`reg_line`	character fit a regression line over the data cloud. Default is linear regression 'lin'. Other options include: - 'q' : quadratic regression; - 'c' : cubic regression; - 'poly' : 3rd degree polynomial expansion.
`reg_weight`	column specify weights for regression, it must be the name of a column of the data.table. Default is NULL.
`x_name`	character Set name for x_axis. Default is NULL
`y_name`	character Set name for y_axis. Default is NULL.
`title`	character Set title of histogram. Default is NULL

Details

tooltype: Analysis

Value

It displays the graph.

Examples

```
g_scatter_reg(myDT,c("yvar","xvyar"),trim=5,both_tails=TRUE,title="My Title")
```

`harmonize_classification` *Remap classifications*

Description

The function remaps the original 'data.table', including a target classification

Usage

```
harmonize_classification(
  DT,
  conc,
  native_class_DT,
  origin_class,
  target_class,
  rm_native_class_DT = F
)
```

Arguments

`DT`	'data.table' including the native_class_DT
`conc`	'character' name of the concordance table mapping native_class_DT to target_class
`native_class_DT`	'character' name of native classification in DT
`origin_class`	'character' name of native classification in concordance table
`target_class`	'character' name of target classification in concordance table

Details

tooltype: Infra

Value

The function returns a 'data.table' including the target classification

Note

Function authored by Mirja Haelbig. Last update: 11/05/2022

`hier_apply` *Hierarchical Aggregation of Data*

Description

Aggregates variables in ‘var_list‘ to unique values of the hierarchical dimensions specified in ‘hhfile‘ by groups. This function allows for aggregation at various hierarchical levels specified by the user. The ‘mdi_aggregate‘ function is utilized for the aggregation process.

tooltype: MDI

Usage

```
hier_apply(
  DT,
  hhfile,
  var_list,
  bygroups,
  hier,
  agg_type = "sum",
  agg_weight = NULL,
  mrg = FALSE,
  disclosure = TRUE
)
```

Arguments

`DT`	The input dataset to be aggregated.
`hhfile`	A dataset containing columns of hierarchy; ‘bygroups[1]‘ must be included in this dataset.
`var_list`	A list of numeric variables in ‘DT‘ whose values are to be aggregated.
`bygroups`	A list of dimensional variables in ‘DT‘ to which the nodes are to be aggregated.
`hier`	Specifies the name of the aggregation hierarchy; if a single node (’h_x’), aggregates ‘h_0‘ to ‘h_x‘ through ‘hhfile‘. If ’ALL’, aggregates ‘h_0‘ to all available nodes.
`agg_type`	The type of aggregation required (options include "sum", "sd", "mean", "median", "aggweighted.mean", "count", "count.nna"); default is "sum".
`agg_weight`	List of variables used to ’weight’ the aggregation operation as defined by ‘agg_type‘; default is NULL.
`mrg`	Controls whether the results of the operation are merged into ‘DT‘; ‘FALSE‘ means ‘DTout‘ contains the result of the operation, ‘TRUE‘ means ‘DTout‘ is merged into ‘DT‘, and ‘DTout‘ is deleted.
`disclosure`	If ‘TRUE‘, dominance criteria and number of observations are added to the output dataset for disclosure control, applicable only if ‘mrg‘ is ‘FALSE‘.

Value

‘DTmg‘, the output dataset containing the aggregated dimensions and variables from ‘var_list‘.

Examples

```
hier_apply(DT, var_list, bygroups = c("nace", "year", "sizeclass"), indHier, hier="h_2", agg_type=c("mean","su
```

`import_data` *Import Data into R data.table*

Description

Reads data from various file formats into an R 'data.table'. This function is a wrapper around multiple file-reading packages such as 'fread' (from 'data.table'), 'haven' (for Stata, SAS, SPSS), and 'openxlsx' (for Excel). It supports CSV, Stata (.dta), Excel (.xlsx), SAS (.sas7bdat), SPSS (.sav), and tab-delimited text files (.txt). You can also specify a list of columns to import and specify which columns should be imported as characters. However, note that for some file formats (e.g., Stata, SAS, SPSS), the function cannot directly import columns as characters during the import process. In such cases, the specified columns are converted to character types **after** the data has been loaded.

Usage

```
import_data(
  dir,
  file,
  format,
  col_list = NULL,
  char_columns = NULL,
  encoding = NULL
)
```

Arguments

`dir`	The directory path where the input file is located.
`file`	The name of the file to be imported.
`format`	The type of the file to be imported. Supported types include: - 'csv' for comma-delimited files (direct import as character supported), - 'txt' for tab-delimited text files (direct import as character supported), - 'dta' for Stata files (post-import conversion to character), - 'xlsx' for Excel files (post-import conversion to character), - 'sas7bdat' for SAS files (post-import conversion to character), - 'sav' for SPSS files (post-import conversion to character).
`col_list`	A character vector containing the names of the columns to import. If 'NULL' (default), all columns are imported.
`char_columns`	A character vector specifying which columns to treat as characters. For file formats where character conversion during import is not supported, the columns will be converted to character type **after** import.

Value

A 'data.table' containing the data from the imported file.

Note

For CSV and TXT files, the columns specified in ‘char_columns‘ are imported as character during the data reading process. For Stata, Excel, SAS, and SPSS files, character columns are converted **after** the data is loaded into R.

Author(s)

EB, Marco Miorandi 14/10/2024

Examples

```
data <- import_data(
  "path/to/directory/",
  "SBS_2014.csv",
  "csv",
  char_columns = c("column1")
)

data <- import_data(
  "path/to/directory/",
  "datafile.dta",
  "dta",
  col_list = c("column1", "column2"),
  char_columns = "column1"
)
```

---

log_warnings *Logging Warning to a txt File*

Description

This file is called by rocket.R and saves all warnings, diagnostics and additional information in a file "WarningDescription.txt" -> this is to be used instead of printing out to the console to information on the code

Usage

```
log_warnings(
  message,
  path = dirOUTPUT,
  append = TRUE,
  file_name = "WarningsDescription.txt"
)
```

Arguments

message	string for the message/warning that will be saved
path	outputdir of where file is stored, defaults to dirOUTPUT defined in the countdown
append	if to append to the file (as opposed to creating it)

make_conc *Harmonizes classification over time*

Description

This tool creates concordance over the all relevant period for the classification of interest, starting from concordance tables. Starting from a Babel tower, history gets harmonized again.

Usage

```
make_conc(conc_table, year_list, code_name)
```

Arguments

conc_table	concordance tables (data.table) for each couple of years;
year_list	years of interest (starting from the first year t in the first concordance table, not from t-1);
code_name	character name of the classification under analysis;

Value

data.table harmonized classification list in long shape;

Notes: - the concordance tables must have code t-1 in the first column, code t in the second column. - the concordance tables must have full history of codes, since they start until they disappear (if) - "D" is a placeholder for the codes that are disappearing in some years before the last year of the data.

Author(s)

Daniele Aglio, 14/08/2025.

Examples

```
conc_table_list <- make_conc(prodcom_conc_list, 2010:2021, "pcc8")
```

make_filelist *Recursive File finder Find files (default: recursively) that fit a pattern*

Description

The function ‘make_filelist‘ returns a data.table with columns: name, extension, path from a list of full path filenames, found starting at the rootdir (recurseively searched if recursive=TRUE that match any of the strings given in the list patternlist tooltype: Programmer

Usage

```
make_filelist(rootdir, patternlist, recursive = T)
```

Arguments

rootdir	character; an absolute or relative directory to start the search for files
patternlist	list; a list of character strings with patterns. At least one must occur in the full file path/name
recursive	boolean: whether or not the search is recursive through the tree (default=TRUE)

Value

A dataset with matching files, split into name, extension and path

---

mdi_aggregate *Generic Aggregation Function*

Description

Aggregates variables from a sub-aggregate level to a higher aggregate level within a ‘data.table‘. Supports multiple aggregation types, including sum, standard deviation, mean, quantiles (25 missing values, number of non-missing values, number of empty strings, number of zeros, number of positive values, and the Herfindahl-Hirschman Index (HHI).

The function can optionally: - apply weights to variables before aggregation, - merge aggregated statistics back into the original dataset, - compute the number of unique firms in the input data, - and apply disclosure control criteria.

tooltype: MDI

Usage

```
mdi_aggregate(
  DT,
  var_list,
  bygroups,
  agg_type = c("sum"),
  weight = NULL,
  mrg = FALSE,
  disclosure = TRUE,
  count_firms = FALSE
)
```

Arguments

DT	A ‘data.table‘ containing the data to aggregate.
var_list	A character vector of variable names to aggregate.
bygroups	A character vector of grouping variables defining the aggregation level.
agg_type	A character vector specifying the type(s) of aggregation to perform. Supported types: ‘"sum"‘, ‘"sd"‘, ‘"mean"‘, ‘"q10"‘,‘"q25"‘, ‘"median"‘, ‘"q75"‘, ‘"q90"‘, ‘"count"‘, ‘"nmiss"‘, ‘"n_nonmiss"‘, ‘"nempty"‘, ‘"nzero"‘, ‘"npos"‘, and ‘"HHI"‘. Default is ‘"sum"‘.
weight	Optional. A variable name to use as weight for calculating weighted aggregates. Default is ‘NULL‘.

mrg	Logical. If 'TRUE', aggregated statistics are merged back into the original dataset as new variables. If 'FALSE', a new aggregated 'data.table' is returned. Default is 'FALSE'.
disclosure	Logical. If 'TRUE' and 'mrg = FALSE', disclosure criteria are applied, adding dominance indicators and number of observations for disclosure control. Default is 'TRUE'.
count_firms	Logical. If 'TRUE', adds a column 'NumFirms' containing the number of unique firms in the input dataset. A firm identifier column ('plantid', 'firmid', 'entid', or 'entgrp') must be present. Default is 'FALSE'.

Value

- If 'mrg = FALSE': An aggregated 'data.table' containing the requested statistics, optionally with disclosure variables and number of firms. - If 'mrg = TRUE': The input 'data.table' with new columns containing the aggregated statistics.

Author(s)

Cindy Jing Chen, 21/08/2020; Additions by EB, 18/01/2024; Mod. df 28/05/2024; Marco Miorandi 22/09/2025 Marco Miorandi 18/03/2026

Examples

```
# Aggregate employment and revenue by nace and year (sum)
DTagg <- mdi_aggregate(DT, c("emp", "rev"), c("nace", "year"), "sum")

# Weighted mean and count, without merging
DTagg <- mdi_aggregate(DT, c("emp", "rev"),
  c("nace", "year"),
  c("mean", "count"),
  weight = "popweight",
  mrg = FALSE,
  disclosure = TRUE
)

# With number of firms included
DTagg <- mdi_aggregate(DT, "rev", "year", "sum", count_firms = TRUE)
```

---

mdi_disagg *Assign Variable Values Across Hierarchical Dimensions*

Description

Assigns values from a category dimension in an input dataset ('DT') to a more disaggregated category dimension, particularly useful when the dimensional variable in the input dataset encompasses multiple hierarchical levels. The function prioritizes assignment from the least aggregated dimension in 'DT'.

tooltype: MDI

Usage

```
mdi_disagg(DT, var_list, sourcecat, dim_o, target_cat, conc_cp)
```

Arguments

DT	The input dataset, which must contain the variables specified in ‘sourcecat‘ and ‘var_list‘.
var_list	A vector of numeric variables in ‘DT‘ whose values are to be disaggregated.
sourcecat	The dimensional variable in ‘DT‘ from which values are to be assigned.
dim_o	A vector of other dimensional variable names that, along with ‘sourcecat‘, uniquely identify records in ‘DT‘.
target_cat	The target category dimension used for disaggregation, which must be included in ‘conc_cp‘.
conc_cp	A data table containing the child-parent hierarchy for the disaggregation process.

Value

A data table (‘dbout‘) with values assigned from ‘sourcecat‘ to ‘target_cat‘ across the specified dimensions.

Examples

```
# Assuming DT, var_list, sourcecat, dim_o, target_cat, and conc_cp are predefined:
dbout <- mdi_disagg(DT, var_list, sourcecat, dim_o, target_cat, conc_cp)
```

---

mdi_estimate_prodfun *Estimate production functions with multiple estimators (panel, by-group)*

Description

`mdi_estimate_prodfun()` is a wrapper that estimates Cobb–Douglas production functions on firm-level panel data using a selectable set of estimators (methods). It is designed for *grid runs* across within-method specifications (e.g., degree choices, time fixed effects, alternative demeaning) and returns standardized outputs that can be pooled across methods and industries.

The wrapper is typically run "by industry" (or another grouping variable) using `bygroup`. Within each group, the function calls one estimator at a time and binds results into a common panel-style output format including:

- firm id and year (or generic `id`, `time`)
- elasticities with standardized names `el_<var>`
- total factor productivity proxy (`tfp`) and optional de-meaned version (`TFP_demeaned`)
- book-keeping: `NumObs`, plus method/spec identifiers if requested

Usage

```
mdi_estimate_prodfun(
  DT,
  methods,
  y,
  endog,
  exog,
  instr = NULL,
```

```
  id,
  time,
  bygroup,
  acf_args = list(),
  lp_args = list(),
  wdrg_args = list(),
  dpgmm_args = list(),
  ols_args = list(),
  cs_args = list(),
  verbose = TRUE,
  drop_empty = TRUE
)
```

Arguments

DT	A data.table or data.frame with firm-level panel data.
methods	Character vector of methods to run. Supported: "acf", "lp", "wdrg", "dpgmm", "ols", "cs".
y	Character scalar. Output variable name (typically log output/value added).
endog	Character vector. Free/endogenous input(s) (e.g., log labor cost).
exog	Character vector. State/exogenous input(s) (e.g., log capital).
instr	Character vector. Proxy/instrument variable(s) (e.g., log materials). Required by "acf", "lp", "wdrg"; ignored by methods that do not use proxies.
id	Character scalar. Firm identifier column.
time	Character scalar. Time identifier column.
bygroup	Optional character scalar. If provided, estimation is performed separately for each group (e.g., industry).
acf_args	Optional list. Tuning arguments for ACF. See Details.
lp_args	Optional list. Tuning arguments for LP. See Details.
wdrg_args	Optional list. Tuning arguments for Wooldridge system-GMM. See Details.
dpgmm_args	Optional list. Tuning arguments for dynamic panel GMM. See Details.
ols_args	Optional list. Tuning arguments for OLS. See Details.
cs_args	Optional list. Tuning arguments for cost shares. See Details.
...	Reserved for future extensions.

Details

Input convention The wrapper assumes all production-function variables are in logs unless a method explicitly requires levels (e.g., cost shares with log_values=TRUE will exponentiate internally).

Minimal required columns in DT are:

- output y
- free/endogenous input(s) endog (e.g., labor cost or employment)
- state/exogenous input(s) exog (e.g., capital)
- proxy/instruments instr (required by ACF/LP/WRDG; not necessarily used by OLS/CS)
- panel identifiers id, time

- grouping variable bygroup (if bygroup is provided)

Data must be a properly sorted panel within each group (id-time ordering). Each estimator performs its own NA filtering on the variables it needs; hence the effective sample can differ by method/spec.

Methods implemented Methods are selected via methods. The wrapper recognizes:

"acf" Ackerberg–Caves–Frazer-style control function estimator (Cobb–Douglas). Uses a first-stage regression of y on a flexible polynomial in inputs and proxy to construct Φ, then a GMM stage imposing moment conditions with lagged inputs. Returns firm-level elasticities and a residual-based productivity proxy (tfp).

"lp" Levinsohn–Petrin-style proxy estimator (Cobb–Douglas). First-stage regression uses a proxy polynomial; second stage identifies state-input elasticities through a productivity law-of-motion approximation. Returns firm-level elasticities and a residual-based productivity proxy (tfp).

"wdrg" Wooldridge (2009) system-GMM estimator for production functions. Implements the stacked-equations GMM approach (two-equation system) with a flexible polynomial approximation in state inputs and proxy, and valid instruments based on lag structure. Returns common (within-group) elasticities and firm-level tfp as the structural residual from the estimated equation. Optionally returns standard errors for elasticities as se_el_<var>.

"dpgmm" Dynamic panel GMM estimator with autoregressive output component. Estimates $y_{it} = \rho y_{i,t-1} + \beta' x_{it} + u_{it}$ using GMM with lagged inputs as instruments up to max_lag_Z. Returns elasticities el_<var>, the dynamic parameter rho, and a dynamic residual tfp = y - $\hat{y}$.

"ols" Pooled OLS baseline with flexible polynomial controls (Cobb–Douglas). Runs lm on y with inputs and polynomial terms; returns el_<var> and tfp = y - X$\hat{\beta}$ (optionally de-meaned). Intended as a descriptive baseline.

"cs" Cost-shares (index-number) approach for Cobb–Douglas. Constructs firm-time cost shares from input expenditures in levels and uses averaged shares (by bygroup and time) as elasticities. Computes an index-style productivity measure tfp and also TFP_demeaned. NOTE: if TFP_demeaned is computed within (bygroup, time), then industry-year means are mechanically zero by construction.

Method-specific optional arguments (xxxx_args) Each method accepts an optional list of tuning arguments passed via a dedicated parameter name: acf_args, lp_args, wdrg_args, dpgmm_args, ols_args, cs_args. Unspecified fields fall back to method defaults.

ACF arguments (acf_args)

spec Character. Currently "cd" only.

degree Integer. Polynomial degree in the first stage and in the law-of-motion approximation.

lower_bound_theta, upper_bound_theta Numeric. Optional bounds for elasticities in optimization (bounds may be inactive depending on optimizer choice).

TFP_demeaned Logical. If TRUE, returns TFP_demeaned = tfp - mean(tfp) by year.

TFP_minuend Character. "Phi" or "y". Defines whether tfp is formed from the first-stage fitted component (Φ) or directly from y.

Omega_estimates Logical. If TRUE, stores the estimated coefficients of the productivity law of motion (as strings/columns, e.g. intercept + slopes). Useful for diagnostics.

time_FE Logical. If TRUE, includes time fixed effects in the first stage. This changes the definition of Φ and can materially affect identification.

LP arguments (lp_args)

spec Character. Currently "cd" only.

degree Integer. Polynomial degree in proxy function and law-of-motion.

lower_bound_theta, upper_bound_theta Numeric bounds for state-input elasticities in the second-stage optimization.

TFP_demeaned Logical. If TRUE, returns TFP_demeaned by year.

TFP_minuend Character. "y" (default) or "Phi"; controls how tfp is computed.

Omega_estimates Logical. If TRUE, returns law-of-motion parameters.

time_FE Logical. If TRUE, includes time FE in first stage.

Wooldridge arguments (wdrg_args)

degree Integer. Polynomial degree in the state input(s) and proxy used in the system-GMM design.

tol Numeric. Linear-solver tolerance used in GMM normal-equation solves; increase if near-singular.

TFP_demeaned Logical. If TRUE, returns TFP_demeaned by year.

TFP_minuend Character. Currently "y" only (TFP is defined as residual from estimated equation).

Dynamic panel GMM arguments (dpgmm_args)

max_lag_Z Integer. Maximum lag order used to build instruments from lagged inputs. Larger values increase instrument count and can worsen finite-sample performance.

lower_rho, upper_rho Numeric. Bounds for rho in optimization. Used to enforce stability; recommended values typically in (-1,1).

TFP_demeaned Logical. If TRUE, returns TFP_demeaned by year.

OLS arguments (ols_args)

spec Character. Currently "cd" only.

degree Integer. Polynomial degree for flexible controls.

TFP_demeaned Logical. If TRUE, returns TFP_demeaned by year.

Cost shares arguments (cs_args)

log_values Logical. If TRUE (default), interprets y, endog, exog as logs and exponentiates before cost-share construction.

Interpretation of tfp tfp is not identical across estimators:

- ACF/LP/WRDG/DPGMM: residual-based firm-level productivity proxy.
- CS: index-number productivity measure. If de-meaned by (bygroup,time), industry-year means are mechanically zero.

Comparing levels across methods is generally not meaningful; comparisons should focus on within-method variation, rankings, or correlations.

Value

A data.table (or data.frame, depending on implementation) with one row per firm-year (or per used observation) and columns:

id, time Panel identifiers.

bygroup Grouping variable if used.

el_<var> Estimated elasticities for each input variable. Naming uses the input column names.

`tfp` Productivity proxy as defined by the method.

`TFP_demeaned` Optional de-meaned tfp. Present if enabled by the method args.

`rho` Only for `"dpgmm"`: estimated autoregressive coefficient.

`se_el_<var>` Only for `"wdrg"` (if implemented): standard errors for elasticities.

`NumObs` Number of observations used in that estimation block/group after filtering.

Author(s)

Marco Miorandi

Examples

```
## Not run:
out <- mdi_estimate_prodfun(
  DT      = DT_prod,
  methods = c("acf","lp","wdrg"),
  y       = "ln_nv",
  endog   = "ln_labor_cost",
  exog    = "ln_capital",
  instr   = "ln_raw_materials",
  id      = "firmid",
  time    = "year",
  bygroup = "h_2",
  acf_args  = list(degree = 3, time_FE = TRUE,  TFP_minuend = "Phi"),
  lp_args   = list(degree = 2, time_FE = FALSE, TFP_minuend = "y"),
  wdrg_args = list(degree = 3, tol = 1e-10, TFP_demeaned = TRUE)
)

## End(Not run)
```

---

mdi_export *Export Data and Log Output Description*

Description

Exports a given data table to a specified format (csv, RDS, txt, dta, xlsx, sas) and adds an entry to an output description file.

Usage

```
mdi_export(
  DT,
  hier = FALSE,
  hhfile = NULL,
  format,
  output_name,
  datasets_used,
  purpose,
  share_0_1,
  zeroes,
  rel_other_output = "",
```

```
  selection,
  export_type = "sum_stat",
  description = " "
)
```

Arguments

DT	A 'data.table' object to be exported.
hier	Boolean flag indicating if there are multiple levels of aggregation in the dataset (TRUE for hierarchical data structures).
hhfile	Optionally, the name of the hierarchy file used for aggregation, relevant when 'hier' is TRUE.
format	Character string specifying the format of the export ("csv", "RDS", "txt", "dta", "xlsx", "sas").
output_name	The name of the file to be created, without the file extension and the country code.
datasets_used	The name of the MD_dataset(s) used for the analysis.
purpose	Describe the research purpose of the analysis.
share_0_1	Shares of 0 or 1 refer to 0 or 100 percent of the group ...
zeroes	If there are zeros, explain why they are not revealing additional information; otherwise suppress.
rel_other_output	Describe how this output relates to other previously exported/requested files.
selection	Describe if results relate to a sample selection or the full sample.
export_type	Type of output: "sum_stat", "reg_tab", or "other".
description	Additional explanation of the output file.

Details

tooltype: MDI

Value

Writes files to disk; returns 'NULL' invisibly.

Examples

```
## Not run:
mdi_export(
  DT = datalist_num_firms[[nm]],
  hier = TRUE,
  hhfile = indHier,
  format = "csv",
  output_name = paste0("Coverage_Number_Firms", nm, sep = "_"),
  datasets_used = c("BR", "BS"),
 purpose = "Determine the number of all non-missing observations per macro industry and year.",
 rel_other_output = "Previously I asked to export the same file but for another dataset, i.e. ITGS",
  selection = "The table was generated using the full sample",
  export_type = "sum_stat",
  description = "Number of firms by datasource, macroindustry and year"
)

## End(Not run)
```

mdi_harmonize *Harmonizes NSI data to MD dataset specifications*

Description

The function mdi_harmonize returns a datatable to MD specification, by changing the NSI variables using the metadata concordance file. The function stacks the 'body' of the functions: mdi_revalue mdi_recode mdi_reclass mdi_redefine mdi_remap see these functions for further details Examples: if a row of the concordance with method=revalue has 'detail' is: log(x), or x*3.14159, or format(as.date(x,"fmt1"),"fmt2") then this function executes DT[,NSI_varname:=log(NSI_varname)] or DT[,NSI_varname:=NSI_varname*3.14159] or DT[,NSI_varname:=format(as.date(NSI_varname,"

tooltype: Infra

Usage

```
mdi_harmonize(DT, concordance)
```

Arguments

DT	the name of the dataset whose variable values will be changed
concordance	is the name of the concordance dataset with rows containting NSI_varname, MD_varname, method, and detail

Value

executes all the changes using the detail for all the various harmonize methods

Author(s)

Eric Bartelsman 2024-05-29

mdi_import_mpnames *Cleaning MPnames*

Description

The function mdi_import_mpnames looks for all files starting with MPnames stored in dir, combines them and removes duplicates. MPnames should be csv files.

tooltype: Programmer

Usage

```
mdi_import_mpnames(dir)
```

Arguments

dir	directory path, the file path of the directory where the MPnames files are stored

Value

final_data, a dataset (data.table) which is a combination of all MPnames.csv files present in the directory (dir)

Author(s)

Leonardo Indraccolo, 03/2024

`mdi_jointdist` *Calculate Joint Distributions*

Description

Computes joint distributions for specified variables within a data table. This tool allows for the calculation of distributional moments (e.g., deciles, quintiles, quartiles) for one or more variables, aggregated by specified groups and potentially hierarchical structures.

tooltype: Analysis

Usage

```
mdi_jointdist(
  DT,
  hhfile,
  qnames,
  var_names,
  moment,
  bygroups,
  hier,
  agg_type,
  prefix = agg_type,
  agg_weight = NULL,
  mrg = FALSE,
  disclosure = TRUE
)
```

Arguments

`DT`	The input data table that includes variables for distribution calculations.
`hhfile`	A data table containing hierarchical information for aggregation.
`qnames`	A vector of names of variables used for calculating distributions.
`var_names`	The name of the variable(s) whose aggregates are computed.
`moment`	The distributional moment to compute for ‘qnames‘. Can be ’decile’, ’quintile’, or ’quartile’.
`bygroups`	Variables used for stratification in the distribution calculation.
`hier`	Specifies the hierarchical level for aggregation. ’ALL’ indicates all levels; otherwise, specify the desired level.
`agg_type`	The type of aggregation to perform (e.g., sum, mean).
`prefix`	A prefix for naming aggregated variables, typically the same as ‘agg_type‘.

agg_weight	Optional; a variable name in ‘DT‘ representing weights for aggregation.
mrg	A boolean indicating whether to merge results back with the original data table. Default is FALSE.
disclosure	A boolean indicating whether to apply disclosure control measures. Default is TRUE.

Value

A data table with the computed joint distributions, including the distributional moments for specified variables, aggregated according to the given criteria.

Examples

```
# Assuming DT is your dataset with variables 'income' and 'expenditure' and 'region' and 'sector' as bygroups
mdi_jointdist(DT, hhfile = hhData, qnames = c("income", "expenditure"), var_names = "income", moment = "quintil
```

---

mdi_key_id_switch *Switch Identifier Level via Aggregation or Disaggregation*

Description

Performs aggregation or disaggregation of a harmonized dataset across hierarchical identifier levels using concordance tables and metadata-driven rules.

The supported identifier hierarchy is:

```
plantid -> firmid -> entid -> entgrp
```

The function relies on ‘MD_varnames‘ to identify keys and variable structure, ‘MD_aggr_disaggr_methods‘ to determine how each variable should be transformed, ‘MD_idInfo‘ to determine country-specific identifier mappings, and concordance tables (‘*_conc.csv‘) to map between identifier levels.

Usage

```
mdi_key_id_switch(DT_name, method, obj_id_type, cols = NULL)
```

Arguments

DT_name	Character. Name of the dataset, for example ‘"BR"‘, ‘"SBS"‘, or ‘"CIS"‘.
method	Character. Either ‘"agg"‘ for aggregation from a lower level to a higher level, or ‘"disagg"‘ for disaggregation from a higher level to a lower level.
obj_id_type	Character. Target identifier level. Supported values include ‘"to_firmid"‘, ‘"to_entid"‘, ‘"to_entgrp"‘, and ‘"to_plantid"‘.
cols	Optional character vector. Subset of columns to process. Default is all columns.

Details

mdi_key_id_switch

The function first loads the harmonized dataset, identifies the current identifier level, loads the appropriate concordance table, and then applies variable-specific aggregation or disaggregation rules.

All transformations are controlled by metadata in ‘MD_aggr_disaggr_methods‘.

For aggregation (‘method = "agg"‘), multiple lower-level units are combined into a higher-level unit, for example from plant to firm. Each variable is aggregated according to its ‘aggregation_method‘.

Supported aggregation methods include ‘"sum"‘, ‘"weighted_avg"‘, ‘"weighted_mode"‘, ‘"mode"‘, ‘"any"‘, ‘"all"‘, ‘"min"‘, and ‘"max"‘.

For disaggregation (‘method = "disagg"‘), higher-level observations are split into lower-level units using concordance mappings, for example from firm to plant. Each variable is transformed according to its ‘disaggregation_method‘.

Supported disaggregation methods include ‘"replicate"‘ and ‘"weighted_alloc"‘.

Aggregation preserves totals for additive variables such as those using ‘"sum"‘. Disaggregation with ‘"replicate"‘ does not preserve totals. Disaggregation with ‘"weighted_alloc"‘ approximately preserves totals. Factor variables are internally converted to character to avoid unintended integer behavior.

Value

A ‘data.table‘ where variables have been aggregated or disaggregated to the requested identifier level.

Examples

```
## Not run:
ener_firm <- mdi_key_id_switch("ENER", "agg", "to_firmid")
cis_plant <- mdi_key_id_switch("CIS", "disagg", "to_plantid")

## End(Not run)
```

---

mdi_mergedatatables *Merge Data Tables with Optional Variable Selection and Memory Management*

Description

Reads two data tables (left and right) and merges them according to specified keys, join type, and variable selection. Optionally 'pads' years to fill gaps in firms' time series if ‘padyear‘ is set to TRUE. Provides an option to remove the original data tables from RAM to conserve memory.

tooltype: MDI

Usage

```
mdi_mergedatatables(
  DT_left,
  DT_right,
  vars_left,
  vars_right,
  mergekey = c("firmid", "year"),
  jointype = "inner",
  padyear = TRUE,
  rm_lr = FALSE
)
```

Arguments

DT_left	The left data table for merging.
DT_right	The right data table for merging.
vars_left	Variables from the left data table to keep in the merged dataset.
vars_right	Variables from the right data table to keep in the merged dataset.
mergekey	A vector specifying the keys for merging the data tables. Default is ‘c("firmid", "year")‘.
jointype	Type of join to perform. Can be "inner", "full", "left", or "right". Default is "inner".
padyear	Logical indicating whether to ’pad’ years in both data tables before merging. Useful for filling gaps in firms’ time series. Default is TRUE.
rm_lr	Logical indicating whether to remove the left and right data tables from RAM after merging. Default is FALSE.

Value

A merged data table based on the specified parameters.

Examples

```
# Assuming DT_left and DT_right are already defined and contain firmid and year variables.
mergedDT <- mdi_mergedatatables(DT_left, DT_right, vars_left = c("var1", "var2"), vars_right = c("var3", "var4
```

---

mdi_outlier *Outlier Routine for Data Cleaning*

Description

Executes a specified outlier handling routine (trimming, winsorizing, or flagging) on a dataset for selected continuous variables. This function can trim or winsorize the data at specified quantiles or flag observations as outliers based on the fraction provided. Trimming replaces outliers with NA, winsorizing replaces outliers with the closest value within the non-outlier range, and flagging marks outliers with a flag variable.

tooltype: MDI

Usage

```
mdi_outlier(DT, var_list, routine, fraction, both_tails = FALSE, group = NULL)
```

Arguments

DT	Input data table.
var_list	A vector of continuous variable names in 'DT' to be processed by the outlier routine.
routine	A string specifying the outlier routine to apply: "trim", "winsorize", or "flag".
fraction	The fraction of data to be trimmed or winsorized; must be a value between 0 and 1.
both_tails	Logical indicating whether to apply the routine to both tails of the distribution. If 'TRUE', the operation affects both the upper and lower tails; otherwise, it affects only the upper tail.
group	An optional variable name for grouping data before applying the outlier routine. Allows for outlier processing within specific groups.
flag	Logical indicating whether to flag observations as outliers instead of trimming or winsorizing them.

Value

A modified copy of 'DT' with the outlier routine applied to the specified variables. If flagging is selected, new columns are added to indicate flagged observations.

Examples

```
DT <- data.table(id = 1:10, income = c(50, 55, 45, 60, 200, 40, 45, 55, 65, 1000))
var_list <- c("income")
cleaned_data <- mdi_outlier(DT, var_list, "trim", 0.1, both_tails = TRUE)
```

---

mdi_regress *Weighted Regression with Automatic Disclosure Check*

Description

Runs regression analysis on a 'data.table' using models from the **fixest** package ('feols', 'feglm', etc.), with automatic disclosure control applied.

Disclosure rules ensure that regressions are only reported if they meet minimum thresholds for sample size. - If 'CountryCode == "DE"': disclosure is based **only** on the number of unique firms in the data. - For all other countries: disclosure is based on both the total number of observations ('nobs') and the residual degrees of freedom.

The function supports clustered standard errors, regression weights, instrumental variables, and optional LaTeX export of results.

tooltype: Analysis

Usage

```
mdi_regress(
  DT,
  formula,
  model = "feols",
  family = NULL,
  vcov = "iid",
  cluster = FALSE,
  tex = FALSE,
  output_name = NULL,
  desc_file = NULL,
  weights = NULL,
  iv = FALSE,
  num_firms = NULL,
  num_ent = NULL
)
```

Arguments

DT	A ‘data.table‘ containing the dataset for regression analysis.
formula	A character vector of regression formulas.
model	Character. The regression model to use (e.g. ‘"feols"‘, ‘"feglm"‘). Default is ‘"feols"‘.
family	Family specification for GLM models (only relevant if ‘model = "feglm"‘). Example: ‘binomial‘.
vcov	Variance-covariance specification for standard errors. Can be ‘"iid"‘ (default) or a formula for clustered SE (e.g. ‘~clustervar‘).
cluster	Logical. If ‘TRUE‘, standard errors are clustered. In this case ‘vcov‘ must specify the clustering variable. Default is ‘FALSE‘.
tex	Logical. If ‘TRUE‘, regression results are exported as a LaTeX table using ‘etable()‘. Default is ‘FALSE‘.
output_name	Character. Name of the LaTeX output file (without extension), if ‘tex = TRUE‘. Default is ‘NULL‘.
desc_file	Character. Name of a description file to which an entry for the regression output is appended (if ‘tex = TRUE‘). Default is ‘NULL‘.
weights	Optional formula specifying the weights variable to apply in the regression (e.g. ‘~myweights‘). Default is ‘NULL‘.
iv	Logical. If ‘TRUE‘, the regression is treated as an instrumental variable model and IV-specific fit statistics are reported. Default is ‘FALSE‘.
num_firms	Optional integer. Number of unique firms in the regression dataset, required if no firm identifier column (‘plantid‘, ‘firmid‘, ‘entid‘, ‘entgrp‘) is present and ‘CountryCode == "DE"‘.

Details

- For Germany: regression output is only returned if ‘NumFirms >= MDIminNumObs‘. - For other countries: regression output is only returned if both ‘nobs >= MDIminNumObs‘ and residual degrees of freedom ‘>= MDIminNumObs‘. - If ‘tex = TRUE‘, results are saved to ‘<dirOUTPUT>/<output_name>.tex‘ and an entry is appended to ‘<dirOUTPUT>/<desc_file>.txt‘.

Value

A ‘data.table‘ of regression results, including: - Coefficients and standard errors - Confidence intervals (‘ci.lower‘, ‘ci.upper‘) - Sample size (‘NumObs‘), residual df (‘df‘), and number of firms (‘NumFirms‘) - Fit statistics (R^2, Adj. R^2, AIC, BIC, LogLik, and IV/F tests if applicable)

If disclosure criteria are not satisfied, the regression is skipped and a message is printed.

Author(s)

Mirja Haelbig, 06/09/2022; Changed 06/2025 Alberto Ferreira; Changed 06/2025 Marcelo P Ribeiro; Changed 09/2025 Marco Miorandi Changed 02/2026 Dominik Frommeyer

Examples

```
# Basic OLS
mdi_regress(DT, formula = "y ~ x1 + x2")

# Weighted GLM with clustered SEs
mdi_regress(DT,
            formula = "y ~ x1 + x2",
            model = "feglm",
            family = binomial,
            weights = ~w,
            cluster = TRUE,
            vcov = "~firmid")

# IV regression with LaTeX output
mdi_regress(DT,
            formula = "y ~ x1 | z1",
            iv = TRUE,
            tex = TRUE,
            output_name = "iv_reg",
            desc_file = "analysis_log")
```

---

mdi_relpath *Return the path relative to the root*

Description

Function used return a path relative to a root directory. This is used to take the path from NSI_datafiles and find the subdir relative to dirINPUTDATA If the root is not the beginning of the path, then the function returns the given path!

Usage

```
mdi_relpath(path, root)
```

Arguments

path	The string containing a windows or unix path name
root	The root of the directory containing given paths

Value

relative path

Author(s)

Eric Bartelsman 18-09-2024

Examples

```
NSI_datafiles[,path:=mdi_relpath(path,dirINPUTDATA)]
```

`mdi_sampleweights` *Calculate Sample Weights*

Description

Calculates sample weights based on the provided parameters. This function: - Excludes rows with missing values in specified variables. - Computes sample weights by dividing the theoretical number of firms by the number of observations in each stratification group. - Ensures that no sample weight is less than one. - This process is aligned with sample reweighting techniques as described in ProdSys documentation.

tooltype: Analysis

Usage

```
mdi_sampleweights(DT, var_list, popweight, bygroups)
```

Arguments

`DT`	A data table containing the data.
`var_list`	A vector of variable names in 'DT' that should not have missing values.
`popweight`	The theoretical number of firms, used as a numerator for calculating weights.
`bygroups`	Variables that define stratification groups for calculating weights.

Value

A modified copy of 'DT' including a new column 'sampleWeight' that contains the calculated sample weights for each observation.

Examples

```
DT <- data.table(firm = 1:100, industry = rep(1:10, each = 10), sales = runif(100))
var_list <- c("sales")
popweight <- 1000
bygroups <- "industry"
DT_weighted <- mdi_sampleweights(DT, var_list, popweight, bygroups)
```

md_cp2hier *Hierarchical Mapping from Child to Parent Nodes*

Description

Performs hierarchical mapping from a specified child node to its parent nodes in a given dataset. This function is useful for hierarchical data structures where each child node may be associated with one or more parent nodes at specified hierarchical levels.

tooltype: MDI

Usage

```
md_cp2hier(DT, hhfile, hlist_c, hlevel)
```

Arguments

DT	A 'data.table' containing the nodes (e.g., 'h_0', 'h_1', 'h_2', ...) and corresponding category codes (e.g., NACE codes).
hhfile	A 'data.table' representing the metadata with columns of hierarchy that map child nodes to parent nodes.
hlist_c	The name of the child node column in 'DT' that will be mapped to its parent nodes. Must be one of the columns in 'hhfile'.
hlevel	An integer vector specifying the level(s) of the parent node(s) relative to the child node. For example, '1L' for the immediate parent, '2L' for the grandparent, etc. Can specify multiple levels to retrieve multiple layers of parentage.

Value

A list containing two elements: 'DTout', the 'data.table' augmented with the corresponding parent node(s), and 'hlist_p', the name(s) of the parent node column(s).

Examples

```
# Assuming DT, hhfile, and indHier are predefined and contain appropriate hierarchical structure:
result <- md_cp2hier(DT, hhfile, "h_1", c(1L, 2L))
```

pim_capital *Estimate Capital Stock Using Perpetual Inventory Method (PIM)*

Description

Estimates capital stock based on the Perpetual Inventory Method (PIM), as outlined in Halle & Mairesse (1995). This function is tailored for data structured as panel data, with indexing based on a firm identifier ('firmid') and a year variable. It involves the following steps: - Reading the specified variables from the input data.table. - Depending on whether the depreciation rate is provided directly or inferred from the asset type, the function calculates the capital stock. - The output is the input data.table augmented with the new capital stock variable.

The function supports different depreciation formats and asset types, offering flexibility in estimating capital stock across various contexts.

tooltype: Analysis

Usage

```
pim_capital(
  DT,
  dataset_name,
  K0 = "K0",
  I = "ni_tan",
  delta = "d_GFCF",
  output_name = "k_new",
  t = "year"
)
```

Arguments

DT	A data.table including the necessary variables.
K0	The column name for the real initial capital stock value. Default is "K_0".
I	The column name for real investment. Default is "ni_eq".
delta	The column name for the depreciation rate. If 'NULL', an asset type must be specified in 'asset_type'. Default is "dep_ME".
output_name	The name of the output variable for the estimated capital stock. Default is "k_new".
t	The name of the time variable. Default is "year".

Value

A modified 'DT' that includes the new capital stock variable.

Author(s)

Version 1.1, 28 June 2021

References

Halle, P., & Mairesse, J. (1995). "Estimation of the Perpetual Inventory Method for Capital Stock".

Examples

```
DT <- data.table(firmid = 1:5, year = 2010:2014, ni_eq = runif(5), dep_ME = runif(5, 0.01, 0.1), g_ME = runif(5))
pim_capital(DT)
```